



**Progressive Education Society's
Modern College of Engineering, Pune-05.**

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of S.Y. B. Tech [AIDS]

Assignment No: 06

Name: Neha B. Nikam
Class: SY BTech (AIDS)
Section: A
Roll Number: 26048
Subject : Operating System

Title / Problem Statement:

Write a program to demonstrate the concept of deadlock avoidance using Banker's algorithm.

Program Input:

```
# Banker's Algorithm Demonstration

# --- 1. System State Initialization ---

NUM_PROCESSES = 5 # Total number of processes (P0 to P4)

NUM_RESOURCES = 3 # Total number of resource types (R0, R1, R2)
```

#Initial available instances of each resource type

Available = [3, 3, 2]

#Maximum resources each process may still request

Max = [

[7, 5, 3], # P0

[3, 2, 2], # P1

[9, 0, 2], # P2

[2, 2, 2], # P3

[4, 3, 3] # P4

]

#Resources currently allocated to each process

Allocation = [

[0, 1, 0], # P0

[2, 0, 0], # P1

[3, 0, 2], # P2

[2, 1, 1], # P3

[0, 0, 2] # P4

```
]
```

```
#Need matrix (Max - Allocation), initialized to zeros
```

```
Need = [[0] * NUM_RESOURCES for _ in range(NUM_PROCESSES)]
```

```
#--- 2. Safety Algorithm (Core of Banker's) ---
```

```
def calculate_need():
```

```
    """Calculates the initial Need matrix."""
```

```
    for i in range(NUM_PROCESSES):
```

```
        for j in range(NUM_RESOURCES):
```

```
            Need[i][j] = Max[i][j] - Allocation[i][j]
```

```
def is_safe(current_allocation, current_available):
```

```
    """
```

```
    Checks if the current state is safe.
```

```
    Returns: A safe sequence (list of process IDs) or None.
```

```
    """
```

```
    work = list(current_available)
```

```

finish=[False] * NUM_PROCESSES

safe_sequence = []


count = 0

while count < NUM_PROCESSES:

    found=False

    for i in range(NUM_PROCESSES):

        #Check 1: Process is not finished

        if not finish[i]:

            #Check 2: Need[i] <= Work

            can_satisfy = all(Need[i][j] <= work[j] for j in
range(NUM_RESOURCES))

            if can_satisfy:

                #Grant resources and release them (simulate process finish)

                for j in range(NUM_RESOURCES):

                    work[j] += current_allocation[i][j]

                finish[i] = True

                safe_sequence.append(f"P{i}")

```

```
found = True
```

```
count += 1
```

```
break # Restart search for an unfinished process
```

```
#Ifnoprocess could be run in a full pass, the state is unsafe
```

```
ifnotfound:
```

```
    returnNone
```

```
returnsafe_sequence
```

```
#---3.Resource Request Logic ---
```

```
defrequest_resources(process_id, request):
```

```
    """
```

```
    Handlesaresource request using the avoidance algorithm.
```

```
    """
```

```
    print(f"\n--- P{process_id} requests: {request} ---")
```

```
    # 1. Check if Request <= Need (must be true)
```

```
if any(request[j] > Need[process_id][j] for j in range(NUM_RESOURCES)):

    print("ERROR: Request exceeds process's declared maximum Need.")

    return False
```

#2. Check if Request $\leq$ Available (must be true to proceed to tentative allocation)

```
if any(request[j] > Available[j] for j in range(NUM_RESOURCES)):

    print("P must wait: Resources not immediately Available.")

    return False
```

#3. Tentative Allocation (simulate the changes)

```
temp_available = list(Available)

temp_allocation = [list(row) for row in Allocation]
```

```
for j in range(NUM_RESOURCES):

    temp_available[j] -= request[j]

    temp_allocation[process_id][j] += request[j]
```

#4. Check for Safe State with the temporary allocation

```
safe_sequence = is_safe(temp_allocation, temp_available)
```

```

if safe_sequence:

    # State is safe: Commit the changes to the global state

    print(f"Request GRANTED. New Safe Sequence: {' -> ' +
          '.join(safe_sequence)}")

    for j in range(NUM_RESOURCES):

        Available[j] -= request[j]

        Allocation[process_id][j] += request[j]

        Need[process_id][j] -= request[j]

    return True

else:

    # State is unsafe: Rollback (deny the request)

    print("Request DENIED. Granting would lead to an UNSAFE state. P must
    wait.")

    return False

```

#---4. Execution / Demonstration ---

```

if __name__ == "__main__":

```

```

calculate_need()

print(f"Initial Need Matrix Calculated.")

print(f"Initial Available: {Available}")


#Check initial system safety

initial_safe_sequence = is_safe(Allocation, Available)

if initial_safe_sequence:

    print(f"\nInitial state is SAFE. Sequence: {' ->'.join(initial_safe_sequence)}")

else:

    print("\nInitial state is UNSAFE. System Halted.")

    exit()


print("\n" + "=" * 50)

print("=== DEADLOCK AVOIDANCE SIMULATION ===")

print("=" * 50)


#SCENARIO 1: Request that leads to a safe state (GRANTED)

request_resources(1, [1, 0, 2])

```

#SCENARIO 2: Request that exceeds available resources (DENIED/WAIT)

#Current available: [2, 3, 0]. P4 needs [4, 3, 1]. Requesting [3, 3, 0]

request_resources(4, [3, 3, 0])

#SCENARIO 3: Request that leads to an unsafe state (DENIED/WAIT)

#P0 requests [0, 2, 0]. Available [2, 3, 0] -> Tentative [2, 1, 0]

request_resources(0, [0, 2, 0])

#SCENARIO 4: Successful request (GRANTED)

request_resources(3, [0, 1, 1])

Program Output:

```
Dynamic-Pro: Home - Go... Home - Go... S.V.B.TechAll Dynamic-Pro: GDB onl... Govinda... dijkstra algo... ASSIGNME... OS Assignm... New Tab +
onlinegdb.com
Lenovo Support Lenovo McAfee Imported from Mic...
signal
---- Calculating Initial Need Matrix ----
Need Matrix (P0, P1, P2):
P0: [7, 4, 3]
P1: [4, 2, 2]
P2: [6, 5, 8]
P3: [5, 1, 1]
P4: [4, 3, 1]

Initial Available Resources: [3, 3, 2]

---- Checking Initial System Safety ----
Initial state is SAFE. Safe Sequence: P1 -> P3 -> P0 -> P2 -> P4

===== DEADLOCK AVOIDANCE SIMULATION START =====

---- Process P1 requests: [1, 0, 2] ----
Resources are available. Performing safety check for tentative allocation...
!!! UNSAFE STATE: No process could be executed in the current pass.
*** The tentative state is UNSAFE. ***
P1 request DENIED. P1 must wait.

---- Process P4 requests: [3, 3, 0] ----
Resources are available. Performing safety check for tentative allocation...
!!! UNSAFE STATE: No process could be executed in the current pass.
*** The tentative state is UNSAFE. ***
P4 request DENIED. P4 must wait.

---- Process P0 requests: [5, 2, 0] ----
Resources are available. Performing safety check for tentative allocation...
!!! UNSAFE STATE: No process could be executed in the current pass.
*** The tentative state is UNSAFE. ***
P0 request DENIED. P0 must wait.

---- Process P3 requests: [0, 1, 1] ----
Resources are available. Performing safety check for tentative allocation...
*** The tentative state is SAFE. ***
P3 request GRANTED. New Safe Sequence: P3 -> P1 -> P0 -> P2 -> P4

...Program finished with exit code 0
Press ENTER to exit console.
Sports headline
Barcelona vs PS...
Search
ENG
IN
16:02
02-10-2025
```



Progressive Education Society's
Modern College of Engineering, Pune-05.

(An Autonomous Institute Affiliated to Savitribai Phule Pune

University) Department of T.Y. B. Tech [AIDS]

Assignment No: 03

Name: Amol Dadarao Rathod
Class: TY BTech (AIDS)
Section: A
Roll Number: 36070
Subject : Computer Vision

Title / Problem Statement:

Illustrate a tool to convert an image specifically from RGB to Grayscale and RGB to HSV color spaces, highlighting the significance of each channel.

Program Input :

```
import cv2  
  
import matplotlib.pyplot as plt  
  
def process_image(image_path):  
    # Step 3: Load the image (OpenCV loads in BGR format by default)  
    image_bgr = cv2.imread(image_path)
```

Step 4: Check if image is loaded

if image_bgr is None:

print("Error: Could not load the image. Please check the file path.")

return

Step 5: Convert BGR to RGB

image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

Step 6: Convert RGB to Grayscale

image_gray = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2GRAY)

Step 7: Convert RGB to HSV

image_hsv = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2HSV)

Step 8: Split HSV channels to show their significance

h_channel, s_channel, v_channel = cv2.split(image_hsv)

Step 9: Display the results

plt.figure(figsize=(12, 8))

Original RGB

plt.subplot(2, 3, 1)

plt.imshow(image_rgb)

plt.title('Original RGB')

plt.axis('off')

Grayscale

```
plt.subplot(2, 3, 2)
plt.imshow(image_gray, cmap='gray')
plt.title('Grayscale (Intensity)')
plt.axis('off')
```

Full HSV (Visualized as RGB for structural differences)

```
plt.subplot(2, 3, 3)
plt.imshow(image_hsv)
plt.title('HSV Color Space')
plt.axis('off')
```

Hue Channel

```
plt.subplot(2, 3, 4)
plt.imshow(h_channel, cmap='gray')
plt.title('Hue Channel (Color Type)')
plt.axis('off')
```

Saturation Channel

```
plt.subplot(2, 3, 5)
plt.imshow(s_channel, cmap='gray')
plt.title('Saturation Channel (Purity)')
plt.axis('off')
```

Value Channel

```
plt.subplot(2, 3, 6)
plt.imshow(v_channel, cmap='gray')
plt.title('Value Channel (Brightness)')
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

Run the function (Replace 'your_image.jpg' with an actual image file)

```
process_image('BMW.jpeg')
```

Program Output :

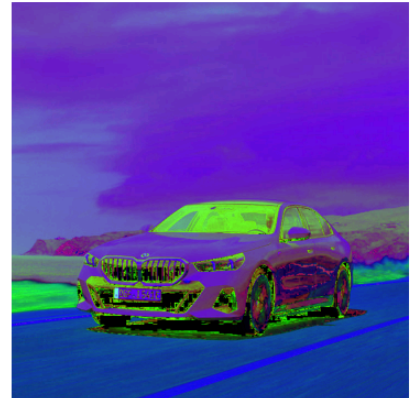
Original RGB



Grayscale (Intensity)



HSV Color Space



Hue Channel (Color Type)



Saturation Channel (Purity)



Value Channel (Brightness)

